



CHUYÊN ĐỀ GAME ONLINE

BÀI 4: ĐỒNG BỘ HÓA THUỘC TÍNH MẠNG VÀ ANIMATION

- ❑ Hiểu cách sử dụng Networked Properties để đồng bộ hóa dữ liệu trong môi trường mạng.
- ❑ Đồng bộ trạng thái của Player (HP, MP, ...).
- ❑ Quản lý Animation của Player để hiển thị nhất quán trên các client.
- ❑ Cập nhật trạng thái Animation bằng Network Properties

Networked Properties là gì?

Là các thuộc tính mạng được tự động đồng bộ hóa giữa tất cả client và State Authority.

Các thuộc tính này được khai báo với [Networked].

Lợi ích:

- ☐ Đảm bảo dữ liệu luôn nhất quán giữa các client.
- ☐ Tiết kiệm băng thông vì chỉ đồng bộ khi có thay đổi.

Lợi ích của Network Properties:

- ☐ Đồng bộ tự động: Không cần viết thêm logic để gửi và nhận dữ liệu.
- ☐ Tiết kiệm băng thông: Chỉ đồng bộ khi giá trị thay đổi.
- ☐ Quản lý dễ dàng: Hỗ trợ trực tiếp các sự kiện thay đổi với `OnChangedRender`.

Cú pháp:

```
[Networked, OnChangedRender(nameof(OnHealthChanged))]  
private int Health { get; set; }
```

- ❑ Đồng bộ giá trị giữa tất cả các client.
- ❑ Giá trị chỉ được thay đổi bởi State Authority.
- ❑ Tích hợp hỗ trợ cho sự kiện thay đổi giá trị bằng OnChangedRender.

Chúng ta tạo class PlayerProperties và gắn vào Player prefab ở bài trước

```
public class PlayerProperties : NetworkBehaviour
{
    [Networked, OnChangedRender(nameof(OnHealthChanged))]
    private int Health { get; set; }
    public Slider healthSlider;
    private void OnHealthChanged()
    {
        healthSlider.value = Health;
        Debug.Log($"Health changed to {Health}");
    }
    void Update()
    {
        if (HasStateAuthority)
        {
            if (Input.GetKeyDown(KeyCode.Space))
            {
                Health -= 10;
            }
        }
    }
}
```

Trong đó

- ❑ [Networked]: Biến Health được đồng bộ giữa các client.
- ❑ OnChangedRender: Gọi hàm OnHealthChanged khi giá trị Health thay đổi.

OnChangedRender là gì?

- ❑ OnChangedRender là một delegate được kích hoạt khi thuộc tính [Networked] thay đổi.
- ❑ Cơ chế này hoạt động trong Render Loop và đảm bảo rằng mọi thay đổi về trạng thái được phản ánh ngay lập tức lên giao diện hoặc hiệu ứng đồ họa.

Cơ chế hoạt động OnChangedRender:

- Khi một thuộc tính [Networked] thay đổi, Fusion sẽ tự động gọi phương thức được liên kết thông qua OnChangedRender lúc khai báo

Lợi ích OnChangedRender:

- ❑ Cập nhật đồ họa mượt mà: Các thay đổi như màu sắc, hiệu ứng hình ảnh được phản ánh ngay khi thuộc tính thay đổi.
- ❑ Tăng hiệu suất: Giảm việc xử lý không cần thiết trong logic game.

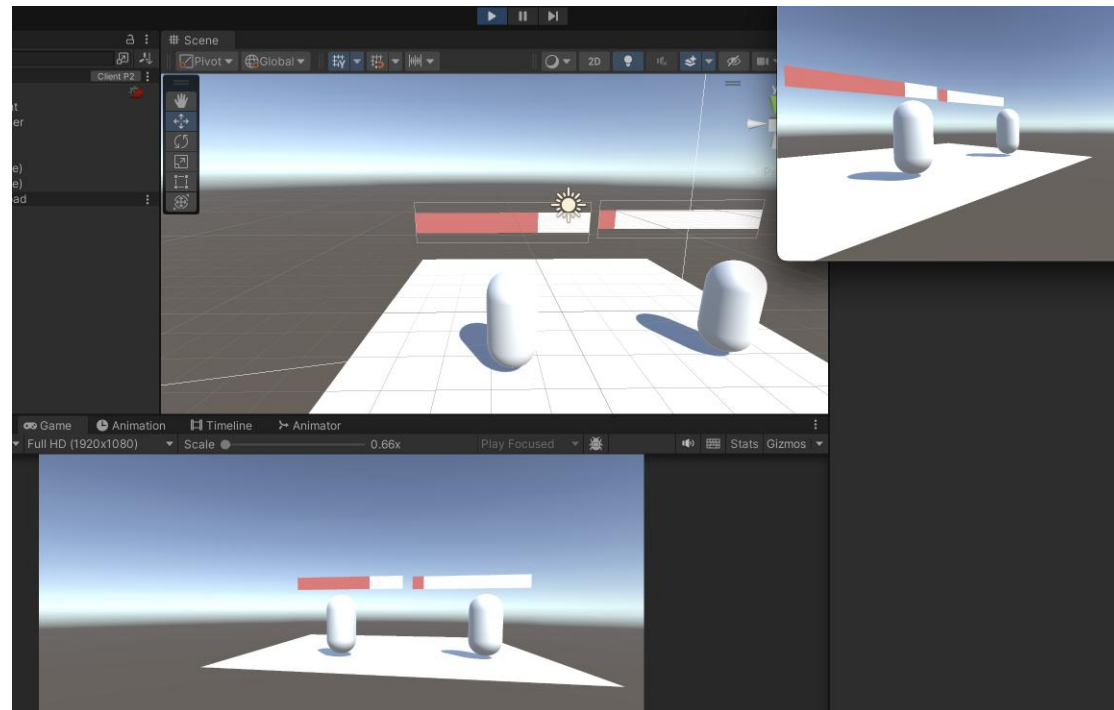
Quay lại đoạn code trên, khi Health thay đổi:

- ☐ Fusion gọi OnHealthChanged.
- ☐ Phương thức OnHealthChanged được thực thi, cập nhật thanh máu trên HUD.

State Authority:

- ☐ Là nguồn duy nhất có quyền thay đổi giá trị [Networked].
- ☐ Health chỉ được cập nhật bởi State Authority, đảm bảo tính công bằng.

Bây giờ chúng ta chạy game ở hai đối tượng sẽ được kết quả như sau



- ❑ Như vậy, thông tin chỉ số Health của nhân vật đã được đồng bộ trong mạng
- ❑ Các HUD hiển thị cũng đã được đồng bộ theo thuộc tính Health

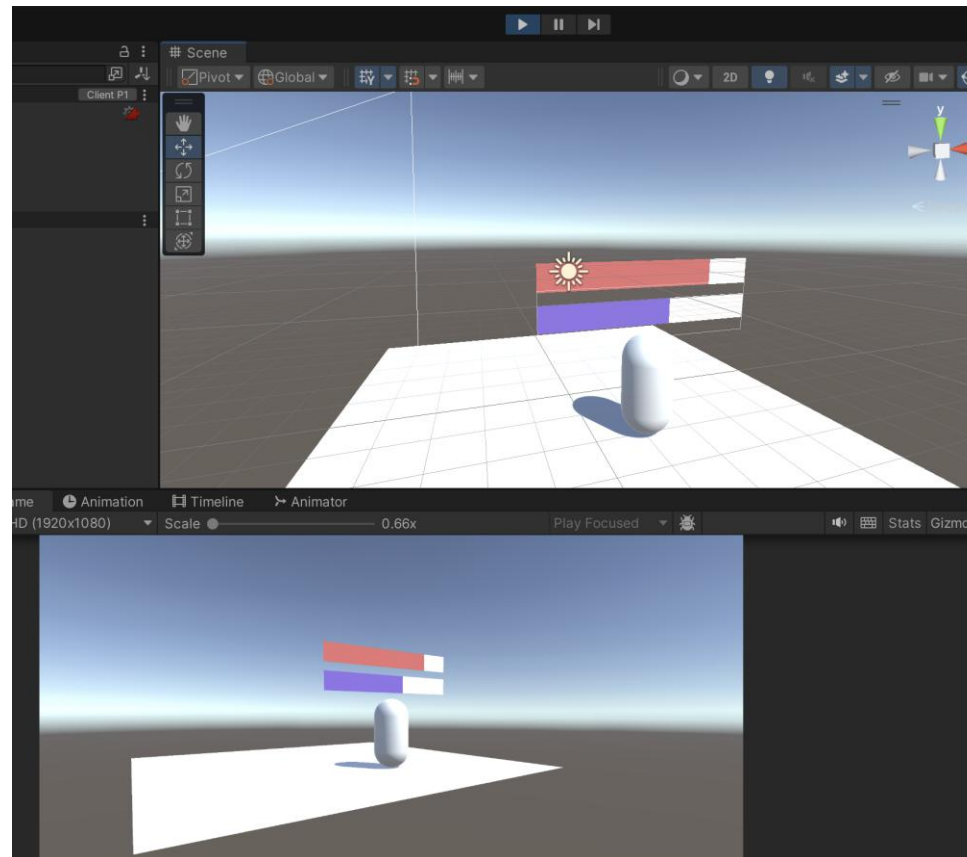
Trong trường hợp chúng ta muốn đồng bộ dữ liệu phức tạp hơn thì nên dùng struct

```
[System.Serializable]
public struct PlayerInfo : INetworkStruct
{
    public int health;
    public int mana;
    public int score;
}
```

Bây giờ, chúng ta điều
chỉnh lại cập nhật nhiều
thông tin của Player
như Health và Mana

```
public class PlayerProperties : NetworkBehaviour
{
    [Networked, OnChangedRender(nameof(OnInfoChanged))]
    private PlayerInfo Info { get; set; }
    public Slider healthSlider;
    public Slider manaSlider;
    private void OnInfoChanged()
    {
        healthSlider.value = Info.health;
        manaSlider.value = Info.mana;
    }
    void Update()
    {
        if (HasStateAuthority)
        {
            if (Input.GetKeyDown(KeyCode.Space))
            {
                var currentHealth = Info.health;
                var currentMana = Info.mana;
                Info = new PlayerInfo
                {
                    health = currentHealth - 10,
                    mana = currentMana - 20,
                    score = 100
                };
            }
        }
    }
}
```


Khi chạy lại game, chúng ta sẽ được giao diện sau



Tối ưu hóa Network Properties

- ☐ Giảm kích thước dữ liệu:
 - ✓ Tránh đồng bộ dữ liệu không cần thiết.
 - ✓ Dùng các kiểu dữ liệu nhỏ (như byte, short thay vì int).
- ☐ Đồng bộ chỉ khi cần thiết: Kích hoạt đồng bộ hóa bằng các sự kiện thay vì liên tục cập nhật.

Đồng bộ Animation trong Photon Fusion 2

Đồng bộ Animation là một phần không thể thiếu trong các trò chơi mạng, giúp hiển thị chính xác trạng thái hành động của nhân vật giữa các client.

Các trạng thái Animation phổ biến cần đồng bộ:

- ☐ Trạng thái chuyển động: Chạy, nhảy, đứng yên.
- ☐ Trạng thái chiến đấu: Tấn công, né tránh, trúng đòn.
- ☐ Hiệu ứng đặc biệt: Kích hoạt kỹ năng, chuyển đổi trạng thái.

Các công cụ hỗ trợ trong Fusion 2:

- ☐ Networked Properties: Đồng bộ trạng thái animation.
- ☐ Animator Parameters: Sử dụng Trigger, Bool, Int, Float để chuyển đổi trạng thái.
- ☐ Lag Compensation: Đảm bảo hành động diễn ra chính xác ngay cả khi có độ trễ.

Tổng quan về đồng bộ Animation

Input Authority: Client có quyền điều khiển sẽ gửi thông tin về trạng thái animation (chạy, nhảy, tấn công) qua mạng.

State Authority: Đảm bảo rằng trạng thái animation được cập nhật chính xác và đồng bộ hóa với tất cả client.

Animator Syncing: Thông qua [Networked] và OnChangedRender, trạng thái animation được phát đến các client khác.

Sử dụng [Networked] để đồng bộ trạng thái

```
public class PlayerMovement : NetworkBehaviour

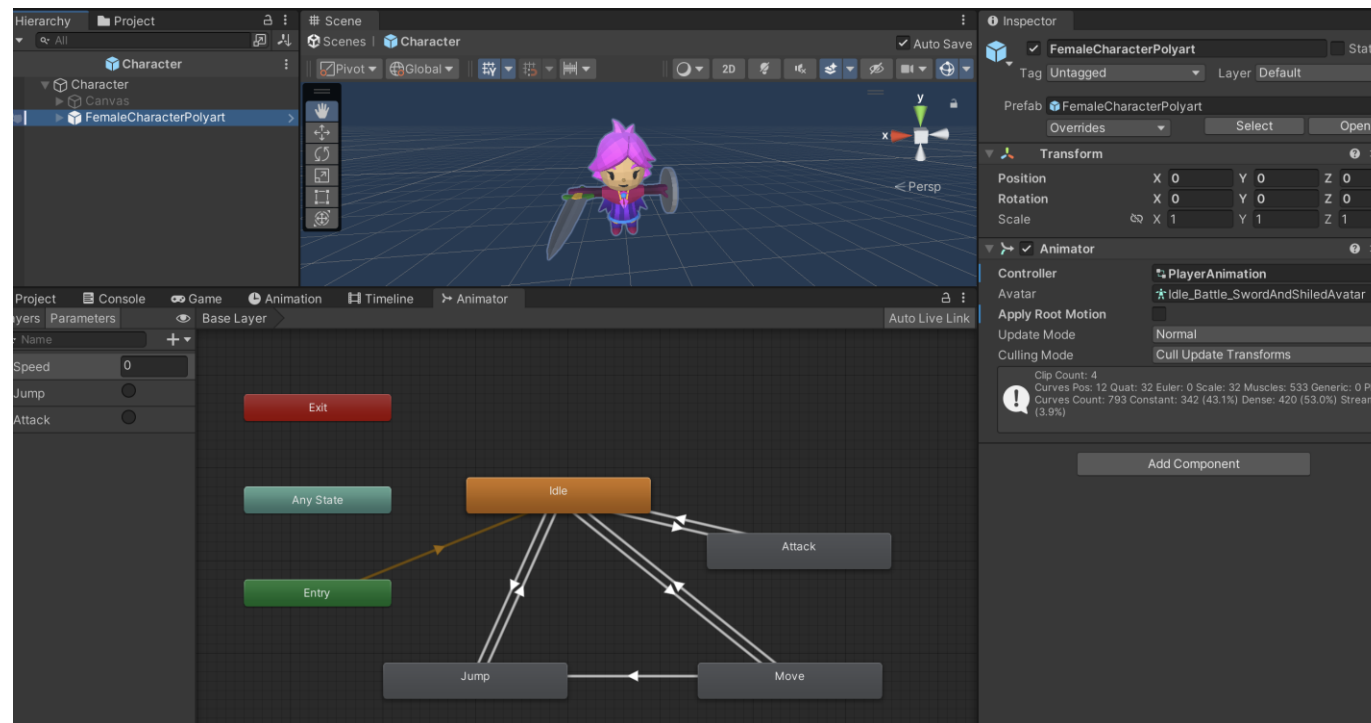
    public Animator animator;
    [Networked, OnChangedRender(nameof(OnSpeedChanged))]
    public float AnimatorSpeed { get; set; }

    void OnSpeedChanged()
    {
        animator.SetFloat("Speed", AnimatorSpeed);
    }

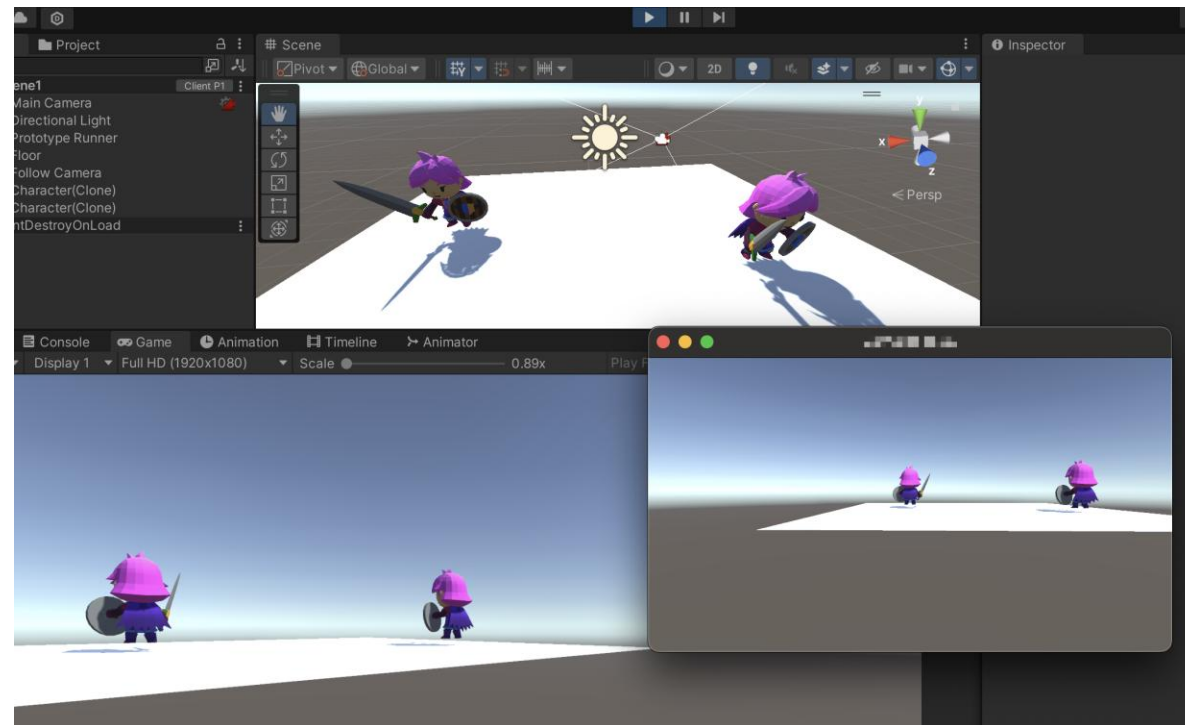
    public override void FixedUpdateNetwork()
    {
        if (!Object.HasInputAuthority) return;
        var horizontal = Input.GetAxis("Horizontal");
        var vertical = Input.GetAxis("Vertical");
        Vector3 move = new Vector3(horizontal, 0, vertical);
        controller.Move(move * speed * Runner.DeltaTime);

        AnimatorSpeed = controller.velocity.magnitude;
    }
}
```


Trong game, chúng ta có thể sử dụng một asset bất kỳ để thực hiện animation như sau



Khi khởi chạy, chúng ta có thể điều khiển thay đổi các trạng thái của animation



Tối ưu hóa đồng bộ Animation

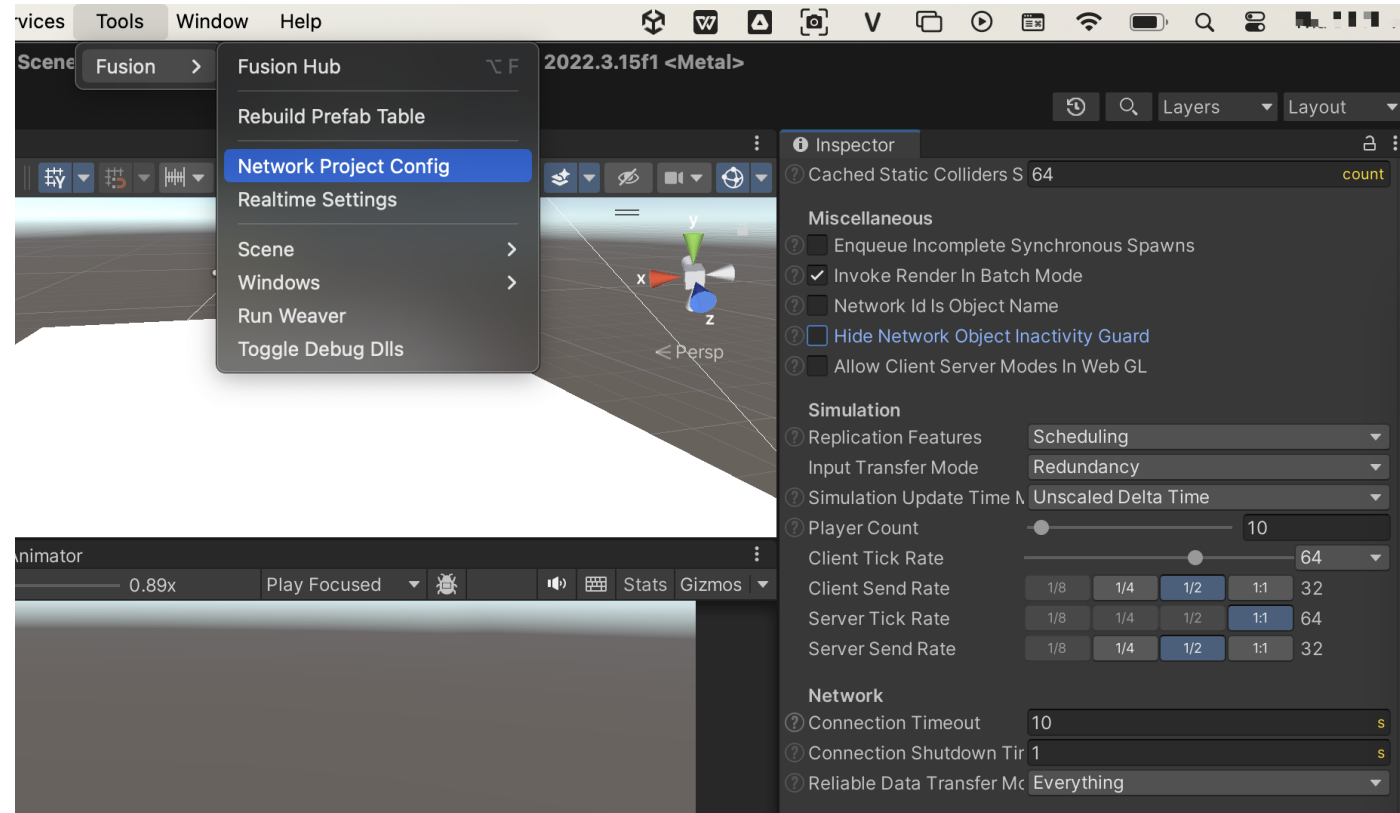
- ☐ Sử dụng Trigger cho Animation ngắn hạn: Nhảy, tấn công, né tránh.
- ☐ Sử dụng Bool cho Animation dài hạn: Chạy, đứng yên.
- ☐ Giảm Tick Rate: Nếu không cần đồng bộ nhanh, hãy giảm tick rate để tiết kiệm băng thông.

- ❑ Chỉ đồng bộ trạng thái quan trọng: Tránh đồng bộ các Animation không cần thiết (ví dụ: trạng thái tĩnh).

Cách giảm Tick Rate

Bạn có thể thay đổi Tick Rate khi khởi động NetworkRunner bằng cách cấu hình thông qua Simulation.TickRate.

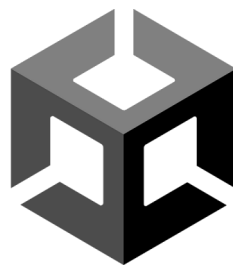
Bằng cách thay đổi thông số trong Simulation, chúng ta có thể thay đổi TickRate



Ví dụ:

Tick Rate = 20: Chỉ cập nhật trạng thái trò chơi 20 lần mỗi giây thay vì 60 lần, giảm đáng kể băng thông sử dụng.

Phù hợp cho các game không yêu cầu đồng bộ nhanh, như game chiến thuật hoặc board game.



Kết thúc